MODULE 5

TIME SERIES ANALYSIS AND PREDICTIVE MODELING

Detailed Study Notes • Python • pandas • Forecasting • Model Evaluation • Sentiment Analysis

Syllabus covered

• Time series: importing, cleaning, plotting, moving averages, missing values, serial correlation, and autocorrelation

• Predictive modeling: overview, evaluation, building a model solution, and sentiment analysis

Beginner-friendly definitions, formulas, worked examples, runnable Python code, revision notes, and exam questions.

How to use these notes

This module has two connected parts. Time series analysis studies data collected over time. Predictive modeling uses past data to estimate an unknown or future outcome. Read the explanations, run the short code examples, and practise explaining what each result means.

Install libraries

```
python3 -m pip install pandas numpy matplotlib scikit-learn statsmodels
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Run code in Jupyter Notebook or a Python file. In a regular Python script, use `plt.show()` to display charts.

Learning outcomes

• Load time-stamped data, clean it, and set a proper datetime index.

• Plot time series and calculate moving averages.

• Understand missing values, serial correlation, and autocorrelation.

• Build a basic predictive-modeling workflow without data leakage.

• Evaluate models with suitable metrics and explain sentiment analysis.

1. Time series analysis: introduction

A **time series** is a sequence of observations recorded in time order, such as daily temperature, monthly sales, hourly electricity use, or yearly population.

1.1 Important components

• **Trend:** a long-term upward or downward movement.

• **Seasonality:** a pattern that repeats at a known interval, such as higher sales every December.

• **Cyclic movement:** longer-term rises and falls without a fixed, exact repeating period.

• **Noise:** irregular variation not explained by the main pattern.

• **Level:** the typical value around which observations may fluctuate.

1.2 Why time order matters

In ordinary tabular data, rows may often be shuffled without changing the task. In time series, order matters: yesterday's value may help predict today's value, and a random train/test split can accidentally use future information to predict the past.

Example: Daily sales may rise during weekends and holidays. A model that ignores date order may miss these patterns or produce an unrealistic evaluation.

2. Importing and cleaning time series data

2.1 Read a CSV and parse dates

```
import pandas as pd

df = pd.read_csv("sales.csv", parse_dates=["date"])
df = df.sort_values("date")
df = df.set_index("date")
print(df.head())
print(df.info())
```

The `parse_dates` argument converts date strings into datetime values. Sorting ensures observations are in chronological order. Setting the date column as the index makes time-based selection and resampling easier.

2.2 Inspect and clean

```
print(df.head())
print(df.tail())
print(df.isna().sum())
print(df.duplicated().sum())
```

```
# Remove duplicate rows only when they are true duplicates
df = df[~df.index.duplicated(keep="last")]

# Convert a measurement to numeric; invalid strings become NaN
df["sales"] = pd.to_numeric(df["sales"], errors="coerce")
```

Do not remove duplicate timestamps blindly: multiple records may legitimately occur at the same time. Decide whether to aggregate them, retain them, or correct them based on what each row represents.

2.3 Set frequency and resample

```
# Daily data: use a daily frequency where appropriate
daily = df["sales"].resample("D").mean()

# Monthly totals, if sales are amounts that should be summed
monthly_total = df["sales"].resample("MS").sum()
```

Resampling groups observations into a new time frequency. Use `mean()` for an average measure, `sum()` for additive totals, or another aggregation that matches the meaning of the variable.

2.4 Time-based selection

```
# All observations from January 2026
jan = df.loc["2026-01"]

# A date range
period = df.loc["2026-01-01":"2026-01-31"]
```

These examples assume a datetime index. Check time zones and timestamp conventions when combining data from different sources.

3. Plotting time series

A time-series line plot puts time on the x-axis and the measured value on the y-axis. It helps reveal trends, seasonality, abrupt changes, and missing periods.

```
df["sales"].plot(figsize=(9, 4), marker="o", linewidth=1)
plt.title("Sales over time")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.grid(True, alpha=0.25)
plt.tight_layout()
plt.show()
```

3.1 What to look for

• Overall direction: upward, downward, or roughly stable.

• Repeating patterns: daily, weekly, monthly, or annual seasonality.

• Sudden jumps or drops: possible events, changes, or data errors.

• Gaps: missing timestamps or missing measurements.

• Changing variability: periods with larger or smaller fluctuations.

3.2 Plot multiple series

```
df[["sales", "cost"]].plot(figsize=(9, 4))
plt.title("Sales and cost over time")
plt.ylabel("Amount")
plt.tight_layout()
plt.show()
```

If series use very different units or scales, use separate panels rather than placing them on one axis without explanation.

4. Moving averages

A **moving average** smooths short-term fluctuations by averaging values in a moving window. It helps reveal the underlying trend but can hide sudden changes.

4.1 Simple moving average (SMA)

For a window of k observations, the simple moving average is the average of the most recent k values. For a 3-point window: MA■ = (x■ + x■■■ + x■■■) / 3.

```
df["MA_7"] = df["sales"].rolling(window=7).mean()
```

```
df[["sales", "MA_7"]].plot(figsize=(9, 4))
plt.title("Sales with 7-observation moving average")
plt.show()
```

The first six values of a 7-point rolling mean are missing by default because a complete seven-observation window is not yet available. Setting `min_periods=1` calculates earlier averages using fewer values, but changes the meaning of the window.

4.2 Centered versus trailing averages

• **Trailing moving average:** uses the current and previous observations; useful for forecasting because it does not use future values.

• **Centered moving average:** centers the window around the time point; useful for describing a trend but may use future observations and is therefore unsuitable as a real-time predictor.

4.3 Choosing a window

A larger window smooths more strongly but reacts more slowly to changes. Choose a window based on the data frequency and the pattern of interest—for example, 7 observations for a weekly pattern in daily data, if that matches the problem.

5. Missing values in time series

Missing values can occur because a sensor fails, data collection stops, or an observation is not recorded. Missing timestamps and missing measurements are different problems: a date may be absent entirely, or the date may exist with a missing value.

5.1 Detect missing values

```
print(df.isna().sum())
print(df["sales"].isna())
```

5.2 Common handling methods

• **Drop:** remove missing rows only if losing those observations is acceptable and does not create bias.

• **Forward fill:** carry the last observed value forward; may suit slowly changing states, but not every measurement.

• **Backward fill:** use the next observed value; can leak future information if used for forecasting features.

• **Interpolation:** estimate values between known observations; useful for some smooth measurements, but can hide genuine abrupt changes.

• **Domain-based imputation:** use a justified rule informed by how the data are collected.

```
df["sales_ffill"] = df["sales"].ffill()
df["sales_interp"] = df["sales"].interpolate(method="time")
```

Time interpolation requires a datetime-like index. Always document the method and consider whether the missingness itself carries information.

5.3 Avoid leakage

When evaluating a forecast, do not fill a training-period gap using values from the test/future period. Fit imputation rules on training data and apply them forward in time.

6. Serial correlation and autocorrelation

Serial correlation means observations in a sequence are related to observations at other times. **Autocorrelation** measures the correlation between a series and a lagged version of itself.

6.1 Lag example

Lag 1 compares each value with the previous value; lag 7 compares a daily observation with the value seven days earlier. Strong lag-7 autocorrelation may suggest a weekly pattern in daily data.

```
# Correlation between sales and the previous observation
lag1_corr = df["sales"].corr(df["sales"].shift(1))
print("Lag-1 correlation:", lag1_corr)
```

6.2 Autocorrelation function (ACF)

```
from statsmodels.graphics.tsaplots import plot_acf

plot_acf(df["sales"].dropna(), lags=30)
plt.title("Autocorrelation of sales")
```

```
plt.show()
```

An ACF plot shows correlation at different lags. Bars outside the plot's confidence bands can indicate autocorrelation beyond what would be expected under a simple no-correlation reference, but interpretation depends on assumptions and sample size.

6.3 Why autocorrelation matters

• Past values may contain useful information for forecasting.

• Ordinary model assumptions often expect errors to be independent; autocorrelated errors can invalidate standard uncertainty estimates.

• Trend and seasonality can create autocorrelation, so inspect and model them appropriately.

• Correlation does not by itself prove a causal relationship.

6.4 Serial correlation versus ordinary correlation

Ordinary correlation often compares two variables measured on the same cases. Autocorrelation compares one variable with its own lagged values across time.

7. Predictive modeling: overview

Predictive modeling uses historical or observed data to estimate an unknown outcome for new cases or future periods. A model learns patterns from examples and applies them to data it has not seen during training.

7.1 Main task types

• **Regression:** predicts a numeric value, such as next month's sales or house price.

• **Classification:** predicts a category, such as spam/not spam or positive/negative sentiment.

• **Forecasting:** predicts future values using time-ordered observations; often a regression-like task with time-specific constraints.

7.2 Typical workflow

• Define the question and target variable.

• Collect data and understand what each row represents.

• Clean data and create useful features.

• Split data into training, validation, and test sets.

• Train a simple baseline and candidate models.

• Evaluate on data not used to fit the model.

• Inspect errors, improve the model, and document limitations.

• Deploy or present the solution and monitor performance over time.

7.3 Features and target

Features (X) are inputs used to predict an outcome. The target (y) is the value or label to predict. For forecasting, features may include past values, day of week, month, holidays, and rolling statistics calculated only from information available at prediction time.

8. Building a predictive-model solution

8.1 Example: predict a numeric value

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Example assumes X contains numeric input columns and y is numeric.
X = df[["feature1", "feature2"]]
y = df["target"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, pred))
print("RMSE:", mean_squared_error(y_test, pred) ** 0.5)
print("R2:", r2_score(y_test, pred))
```

This generic example is for independent rows. For time series, do not randomly shuffle the observations. Split chronologically so training data come before test data.

8.2 Time-based train/test split

```
df = df.sort_index()
split = int(len(df) * 0.8)

train = df.iloc[:split]
test = df.iloc[split:]

# Example naive forecast: each test value is predicted
# using the last value observed in the training set.
last_value = train["sales"].iloc[-1]
pred = np.repeat(last_value, len(test))
mae = np.mean(np.abs(test["sales"].to_numpy() - pred))
print("Naive forecast MAE:", mae)
```

A naive baseline is valuable: if a complex model cannot beat a simple, reasonable baseline on held-out data, it may not be useful.

8.3 Avoid data leakage

• Never use future target values to create features for earlier dates.

• Calculate rolling means with past values only for real-time forecasting; `shift(1).rolling(...)` is often appropriate for a one-step-ahead feature.

• Fit scalers, encoders, and imputers on training data only.

• Do not select a model by repeatedly checking the final test set.

```
# Example of a past-only rolling feature for forecasting
df["past_7_mean"] = df["sales"].shift(1).rolling(7).mean()
```

9. Evaluating predictive models

Evaluation measures how well a model predicts data it did not use for training. The metric must match the type of target and the real-world cost of errors.

9.1 Regression metrics

• **MAE (Mean Absolute Error):** average absolute difference between actual and predicted values. Easy to interpret in target units.

• **MSE (Mean Squared Error):** average squared error; penalizes large errors more strongly.

• **RMSE:** square root of MSE; returns to the target's units.

• **R^2:** compares squared error with a mean-based reference; it can be negative on test data when predictions are worse than that reference.

Lower MAE, MSE, and RMSE are generally better when evaluated on the same target and dataset. Higher R^2 is often better, but none of these metrics alone proves a model is appropriate.

9.2 Classification metrics

• **Accuracy:** fraction of predictions that are correct.

• **Precision:** among predicted positives, fraction that are truly positive.

• **Recall:** among actual positives, fraction detected.

• **F1-score:** harmonic mean of precision and recall.

• **Confusion matrix:** counts true positives, true negatives, false positives, and false negatives.

Accuracy can be misleading when classes are imbalanced. For example, if 95% of emails are not spam, a model that always predicts “not spam” has 95% accuracy but detects no spam.

9.3 Classification evaluation code

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# y_test contains true labels; class_pred contains predicted labels
print("Accuracy:", accuracy_score(y_test, class_pred))
print(confusion_matrix(y_test, class_pred))
print(classification_report(y_test, class_pred))
```

9.4 Cross-validation

Cross-validation evaluates a model across multiple train/validation splits. For time series, use a time-aware method such as `TimeSeriesSplit` so a fold's training data precede its validation data.

```
from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=5)
for train_idx, test_idx in tscv.split(X):
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
    # Fit the model on X_train, y_train; evaluate on test fold
```

10. Sentiment analysis

Sentiment analysis is a natural language processing (NLP) task that identifies the opinion or emotional tone expressed in text. Common labels are positive, negative, and neutral.

10.1 Applications

• Analyzing product or app reviews.

• Summarizing customer feedback.

• Monitoring opinions about a service or campaign.

• Classifying feedback tickets for further review.

10.2 Basic workflow

• Collect text and define sentiment labels or the task.

• Clean text carefully: handle missing entries, URLs, punctuation, casing, and language-specific details as needed.

• Convert text into features, such as TF-IDF vectors or learned embeddings.

• Train a classifier or use a suitable pretrained language model.

• Evaluate with precision, recall, F1, and a confusion matrix.

• Inspect errors, especially sarcasm, negation, mixed opinions, and domain-specific language.

10.3 Simple sentiment classifier with scikit-learn

```
# Install: python3 -m pip install scikit-learn
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

# CSV should contain columns: text, sentiment
reviews = pd.read_csv("reviews.csv")
reviews = reviews.dropna(subset=["text", "sentiment"])

X_train, X_test, y_train, y_test = train_test_split(
    reviews["text"], reviews["sentiment"],
    test_size=0.2, random_state=42, stratify=reviews["sentiment"]
)

model = make_pipeline(
    TfidfVectorizer(ngram_range=(1, 2), min_df=1),
    LogisticRegression(max_iter=1000)
```

```
)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(classification_report(y_test, pred))
```

TF-IDF gives higher weight to words that are informative in a document relative to the collection. A logistic-regression classifier learns associations between these text features and the sentiment labels.

10.4 Sentiment challenges

• **Negation:** “not good” differs from “good”.

• **Sarcasm:** “Great, another delay” may be negative despite a positive word.

• **Context:** “unpredictable” may be negative for a service but positive for a story.

• **Imbalanced labels:** one sentiment may dominate the dataset.

• **Language and domain:** slang, spelling, Telugu-English mixing, and specialized terms may need tailored data or models.

• **Privacy and bias:** use text responsibly and check whether errors affect groups differently.

11. Common mistakes

• Not parsing dates or sorting observations before analysis.

• Using a random split for forecasting and accidentally training on the future.

• Treating every missing value the same way without understanding why it is missing.

• Using a centered moving average as if it were available in real time.

• Assuming autocorrelation proves causation.

• Reporting a model score without a baseline or test-set description.

• Choosing accuracy alone for a highly imbalanced classification problem.

• Allowing test-set information to influence preprocessing or model selection.

• Assuming sentiment models always understand sarcasm or context.

12. Quick revision sheet

Term	Meaning
Time series	Observations recorded in time order.
Trend	Long-term direction of a series.
Seasonality	Pattern repeating at a regular interval.
Moving average	Average over a sliding window used to smooth data.
Missing value	Unobserved measurement or absent timestamp.
Autocorrelation	Correlation between a series and its lagged values.
Regression	Prediction of a numeric target.
Classification	Prediction of a class or category.
MAE / RMSE	Regression error metrics; lower is usually better.
Precision / Recall / F1	Metrics for classification quality.
Data leakage	Using information unavailable at prediction time.
Sentiment analysis	Classifying the opinion or tone in text.

13. Exam practice questions

1. Define time series analysis and explain trend and seasonality.

2. How do you import a CSV file and set a date column as the index?

3. Why must time-series data be sorted chronologically?

4. What is a moving average? Differentiate trailing and centered moving averages.

5. List four methods for handling missing values and one risk of each.

6. Define serial correlation and autocorrelation.

7. What does an ACF plot show?

8. Define predictive modeling. Differentiate regression and classification.

9. Describe the steps involved in building a predictive model.

10. Why should forecasting data be split chronologically?

11. Explain MAE, RMSE, and R^2.

12. Differentiate accuracy, precision, recall, and F1-score.

13. What is data leakage? Give a time-series example.

14. Define sentiment analysis and list its applications.

15. Explain the TF-IDF plus logistic regression workflow for sentiment classification.

Short answer hints

1. Time-ordered observations; trend is long-term direction, seasonality repeats at a regular interval. 2. Use read_csv(parse_dates=...) and set_index(). 3. To preserve time patterns and prevent future-to-past leakage. 4. Sliding-window average; trailing uses current/past, centered may use future values. 5. Drop, forward fill, backward fill, interpolate; each can lose data, propagate stale values, leak future information, or smooth real changes. 6. Correlation across time lags. 7. Correlations at different lags. 8. Numeric target versus categorical target. 9. Define target, collect/clean, feature engineering, split, train, evaluate, improve, document/deploy. 10. Future observations must not inform training. 11. Average absolute error, square-root mean squared error, and comparison with a mean baseline. 12. Overall correctness, correctness among predicted positives, detection among actual positives, and harmonic mean. 13. Using future data in a feature or imputation. 14. Identifying text tone; reviews, feedback, and customer support. 15. Convert text to TF-IDF features, fit classifier, and evaluate on held-out data.

End of Module 5 notes. For exam preparation, practise the definitions, explain each plot or metric in plain language, and understand why time order and leakage matter.